

TP — Visualisation des fonctions d'activation (NumPy + Matplotlib)

But : implémenter et visualiser 4 fonctions d'activation (ReLU, LeakyReLU, Sigmoid, Tanh) avec **NumPy** et **Matplotlib**, puis produire :

1. un **tracé comparatif** (toutes les courbes sur une figure)
2. une **grille de subplots 2×2** (une courbe par sous-figure)

1) Objectifs pédagogiques

À la fin de ce TP, vous serez capable de :

- créer un vecteur d'échantillonnage avec `np.linspace`
- coder des fonctions vectorisées (sans boucles Python)
- organiser des fonctions dans un dictionnaire et le parcourir avec `.items()`
- tracer proprement avec `matplotlib.pyplot` (titre, axes, légende, grille)
- utiliser des **subplots** et personnaliser le style (couleur, marqueur, taille, `markevery`)
- interpréter les formes et propriétés des activations

2) Pré-requis

- Python 3.x
- Bibliothèques : `numpy` , `matplotlib`

3) Consignes générales

- Travaillez en **vectorisé** (NumPy), pas de boucle sur les points de `x` .
- Les dictionnaires `activations` , `couleurs` , `marques` doivent partager **les mêmes clés**.
- Les figures doivent être lisibles : **titre, axes, grille, légende**.

Fiche mémo (rappels)

I. Dictionnaires (dict)

- Créer : `d = {"k": v}`
- Accéder : `d["k"]`
- Parcourir : `for k, v in d.items(): ...`

II. NumPy (utilisé ici)

- `np.linspace(a, b, n)` : n valeurs entre a et b

- `np.maximum(0, x)` : max élément par élément
- `np.exp(x)` : exponentielle élément par élément
- `np.tanh(x)` : tanh élément par élément
- `np.where(cond, A, B)` : sélection élément par élément

III. Matplotlib.pyplot (utilisé ici)

- `plt.figure()` : nouvelle figure
- `plt.plot(x,y, label=..., color=..., marker=..., markersize=..., markevery=...)`
- `plt.title(...)`, `plt.xlabel(...)`, `plt.ylabel(...)`
- `plt.grid(True)`, `plt.legend()`, `plt.show()`

IV. Subplots

- `fig, axes = plt.subplots(2,2, figsize=(...), sharex=..., sharey=...)`
- `axes = axes.ravel()` : aplatit la grille en 1D pour boucler
- `ax.plot(...)`, `ax.set_title(...)`, `ax.set_xlabel(...)`, `ax.grid(True)`, etc.
- `plt.tight_layout()` : évite le chevauchement des textes

Partie A — Mise en place

A1) Imports et vecteur `x`

✓ À faire

- Importer `numpy` et `matplotlib.pyplot`
- Créer `x` avec **600 points** entre `-3` et `3`

🧠 Cellule Python (à compléter)

In []: `# TODO: créer x`

A2) Dictionnaire des fonctions d'activation

✓ À faire

Compléter le dictionnaire `activations` en suivant les formules suivantes :

- **Relu** :

$$f(x) = \max(0, x)$$

- **Sigmoid** :

$$f(x) = \frac{1}{1 + e^{-x}}$$

- **Tanh** :

$$f(x) = \tanh(x)$$

- **LeakyReLU** (avec (a=0.1)) :

$$f(x) = \begin{cases} x & \text{si } x > 0 \\ ax & \text{sinon} \end{cases}$$

Cellule Python (à compléter)

```
In [ ]: activations = {
    "Relu":      None,    # TODO
    "Sigmoid":   None,    # TODO
    "Tanh":      None,    # TODO
    "LeakyRelu": None,    # TODO
}
```

A3) Dictionnaires de style (couleurs + marqueurs)

À faire

- Définir une couleur par fonction
- Définir un marqueur par fonction

Palette de couleurs possibles (Matplotlib)

Vous pouvez utiliser :

- Couleurs de la palette **Tableau** (recommandées) :

```
["tab:blue", "tab:orange", "tab:green", "tab:red", "tab:purple",
"tab:brown", "tab:pink", "tab:gray", "tab:olive", "tab:cyan"]
```

- Couleurs "simples" :

```
["blue", "red", "green", "black", "gray", "orange", "purple", "pink",
"brown", "cyan"]
```

Remarque : il n'existe pas "tab:black" . Pour le noir, utilisez "black" ou "k" .

Liste de marqueurs possibles (Matplotlib)

- "o" : cercle
- "s" : carré
- "^" : triangle haut
- "v" : triangle bas
- "D" : losange

- "x" / "+" : croix
- "*" : étoile
- "." : point

🧠 Cellule Python (à compléter)

```
In [ ]: couleurs = {
    "Relu": None,      # TODO
    "Sigmoid": None,  # TODO
    "Tanh": None,      # TODO
    "LeakyRelu": None, # TODO
}

marques = {
    "Relu": None,      # TODO
    "Sigmoid": None,  # TODO
    "Tanh": None,      # TODO
    "LeakyRelu": None, # TODO
}
```

Partie B — Tracé comparatif (1 seule figure)

✅ À faire

Tracer les 4 activations sur **le même graphique**, avec :

- plot
- label, color, marker, markersize, markevery
- title, xlabel, ylabel, grid, legend, show

🧠 Cellule Python (à compléter)

```
In [ ]: # TODO: tracer toutes les courbes sur une seule figure
```

Partie C — Subplots (grille 2×2)

✅ À faire

- Créer une grille 2×2 avec plt.subplots(2,2, ...) avec: figsize, sharex, sharey
- Aplatir axes avec axes.ravel()
- Tracer une activation par subplot avec ax.plot(...)
- Ajouter titre, labels, grille, légende dans chaque axe
- Utiliser plt.tight_layout() et plt.show() pour ajuster et afficher la figure

🧠 Cellule Python (à compléter)

```
In [ ]: # TODO: tracer les courbes sur une grille 2x2
fig, axes = plt.subplots(2, 2, ...)
```

```
In [ ]: # TODO: comparer plusieurs valeurs de a
```

Extension — Ajouter deux fonctions d'activation supplémentaires

Ajouter les deux fonctions d'activation suivantes :

- **Softplus** :

$$\text{softplus}(x) = \ln(1 + e^x)$$

- **ELU** (avec $\alpha=1$) :

$$\text{elu}(x) = \begin{cases} x & \text{si } x > 0 \\ \alpha(e^x - 1) & \text{sinon} \end{cases}$$

et les inclure dans les deux tracés (comparatif + subplots).

Cellule Python (bonus)

```
In [ ]: # TODO: ajouter softplus dans activations + couleurs + marques
```

Livrables

- 1 fichier `.ipynb` (code commenté)
- 4 figures exportées :
 - `activations_all.png`
 - `activations_subplots.png`
 - `activations_all_extension.png`
 - `activations_subplots_extension.png`